

Linguagem de Programação

Aula 04: Conceitos Básicos em Python

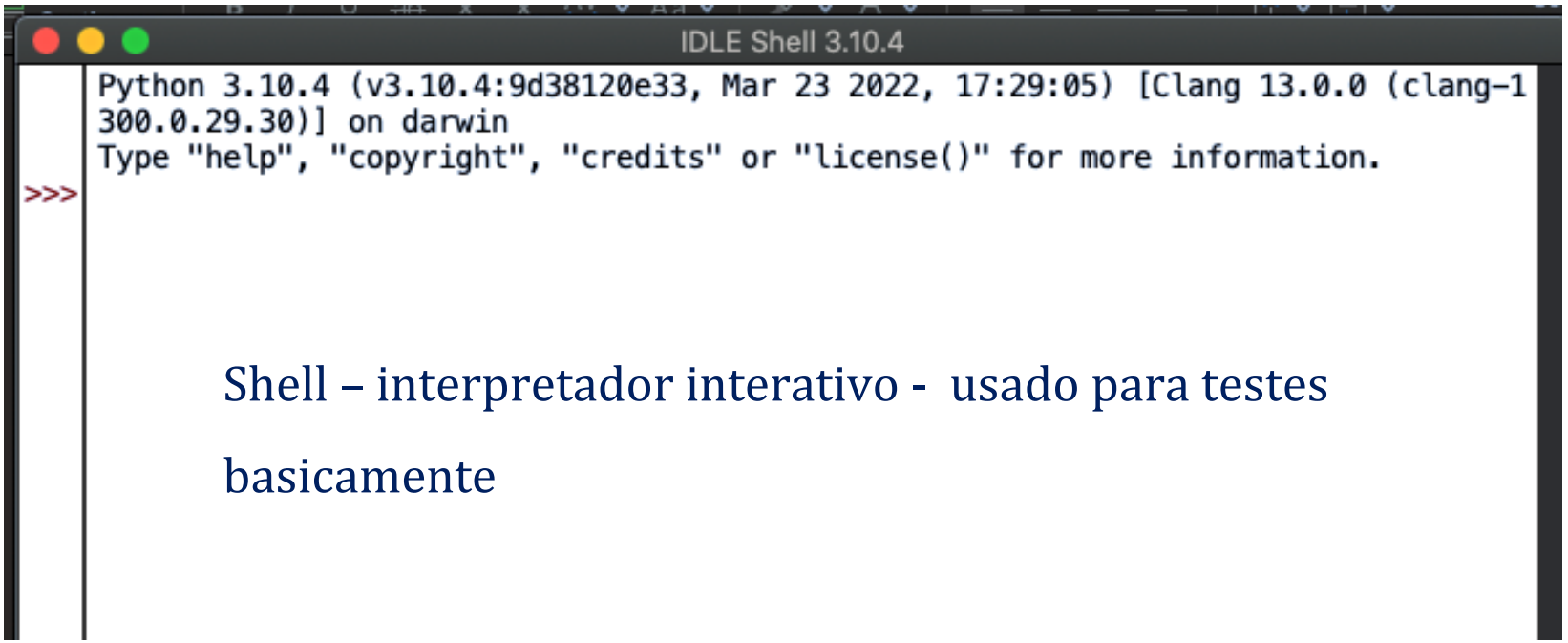
Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Por que aprender Python?

- Simplicidade e clareza
- É um linguagem poderosa e objetiva
- Tem uma grande variedade de aplicações, desde sua utilização em administração de sistemas até o desenvolvimento de grandes projetos
- Muitas bibliotecas para várias aplicações
- Áreas que utilizam Python: IA, BD, biotecnologia, animação 3D, aplicativos móveis, jogos, análise de dados, cálculo numérico
- Python é software livre e extremamente portável
- Os programas podem ser escritos de maneira mais simples do que em outras linguagens

Usando o interpretador Python

- IDLE (Integrated Development and Learning Environment)
  - interface gráfica para interpretar o Python
  - permite também a edição e execução de programas



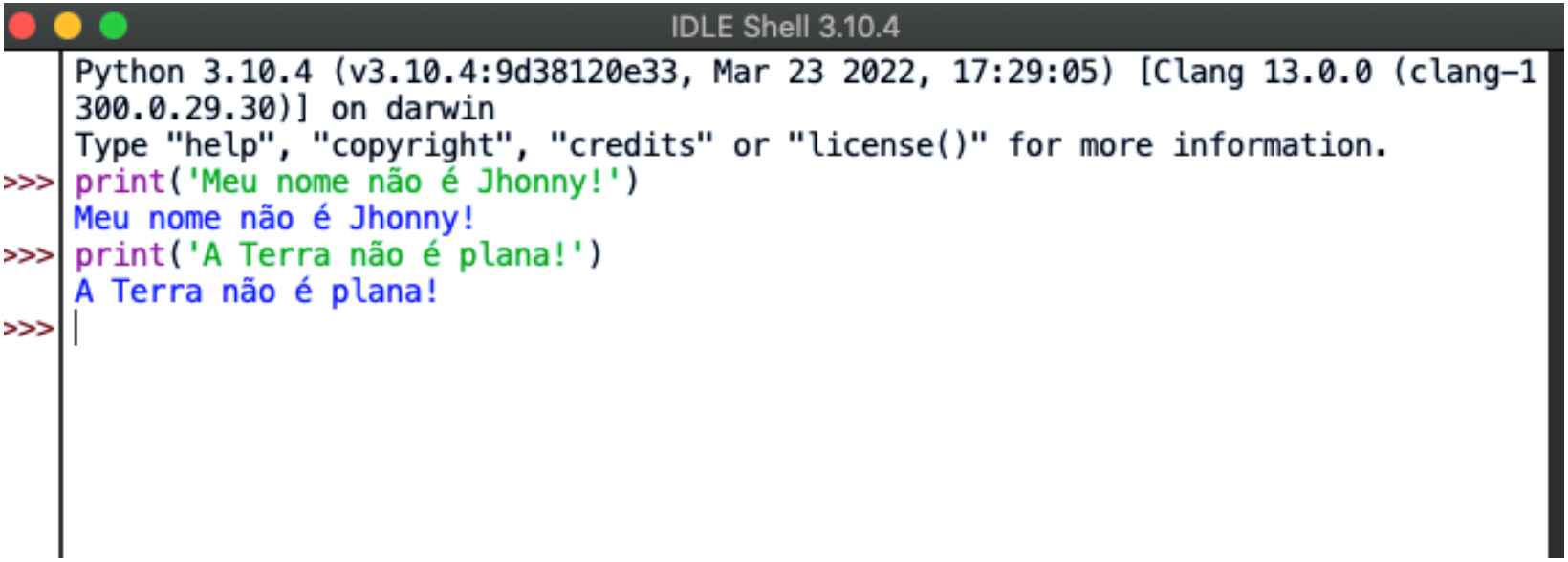
Começando: O interpretador Python

Algumas instruções básicas no shell:

- A cada comando dado o programador deve apertar a tecla [ENTER] para executá-lo
- Para regressar a última instrução dada: [ALT]+[P]
- Para avançar para a próxima instrução inserida: [ALT]+[N]
- Para interromper a execução [CTRL]+[C]

Usando o interpretador Python

Exemplo:



Usando o interpretador Python

Exercício 1: Execute mais instruções no Shell com quaisquer expressões. Utilize os atalhos para verificar se consegue regressar e avançar nas instruções inseridas.

Exercício 2: Refaça o que fez trocando os apóstrofos ‘ por aspas “ e depois elimine as aspas. O que aconteceu?

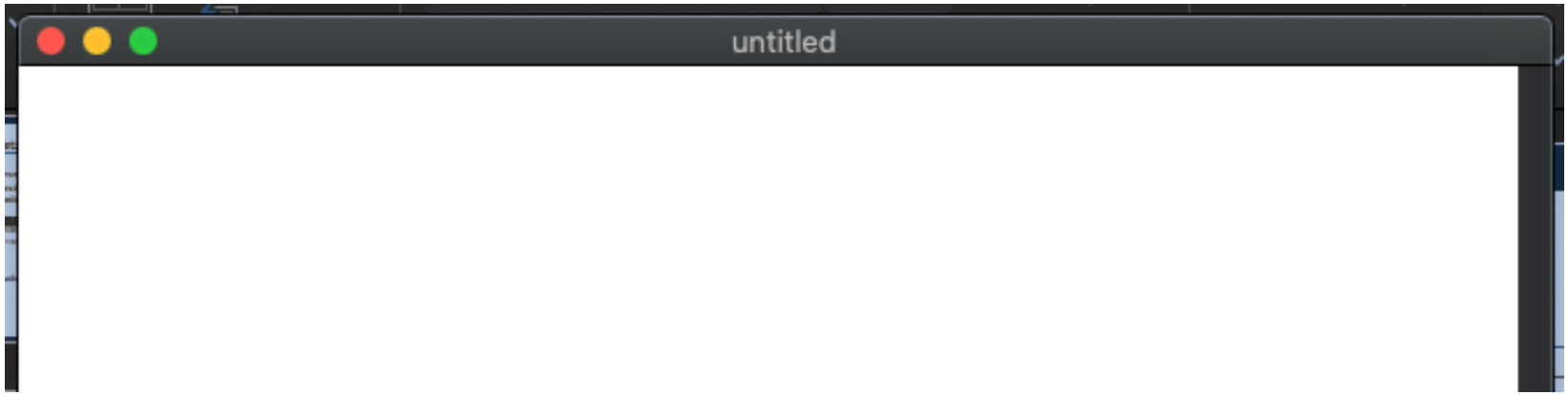
Conclusão: Qualquer texto deve estar entre apóstrofos ou aspas

Usando o interpretador Python

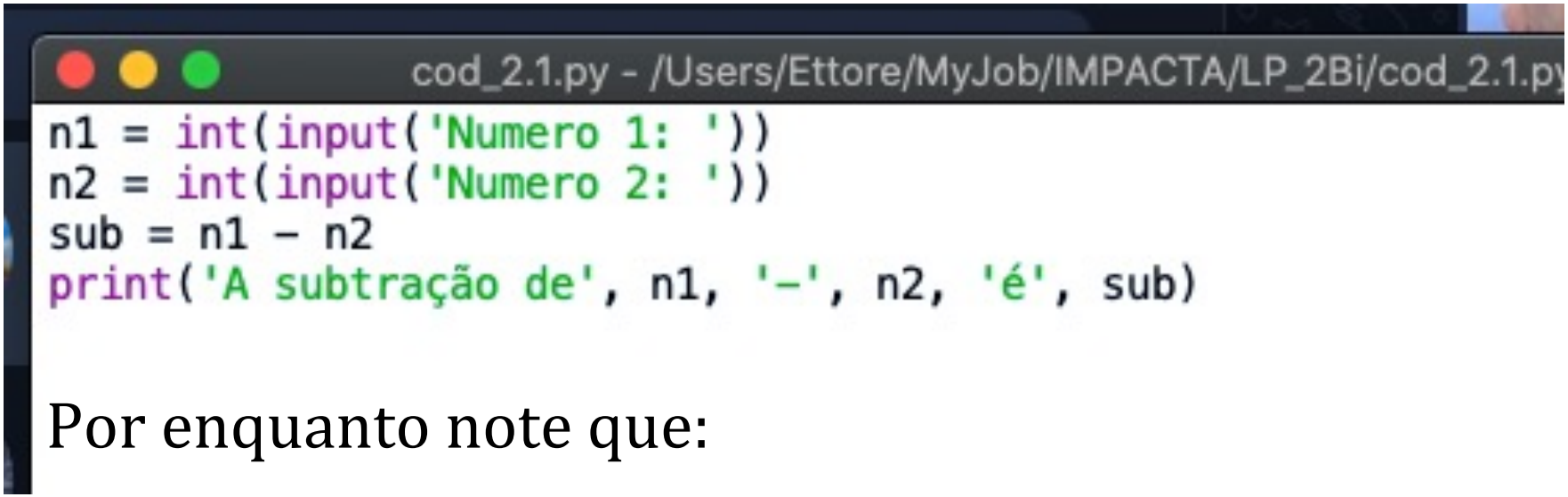
O editor de código fonte

No Shell não é possível salvar o trabalho feito. Logo, quando estamos trabalhando em algum código é necessário ter um ambiente no qual seja possível salvá-lo e editá-lo quantas vezes forem necessárias. No IDLE existe o editor de código fonte.

Para acessá-lo clique no meu em File> New File.



Editor de código fonte



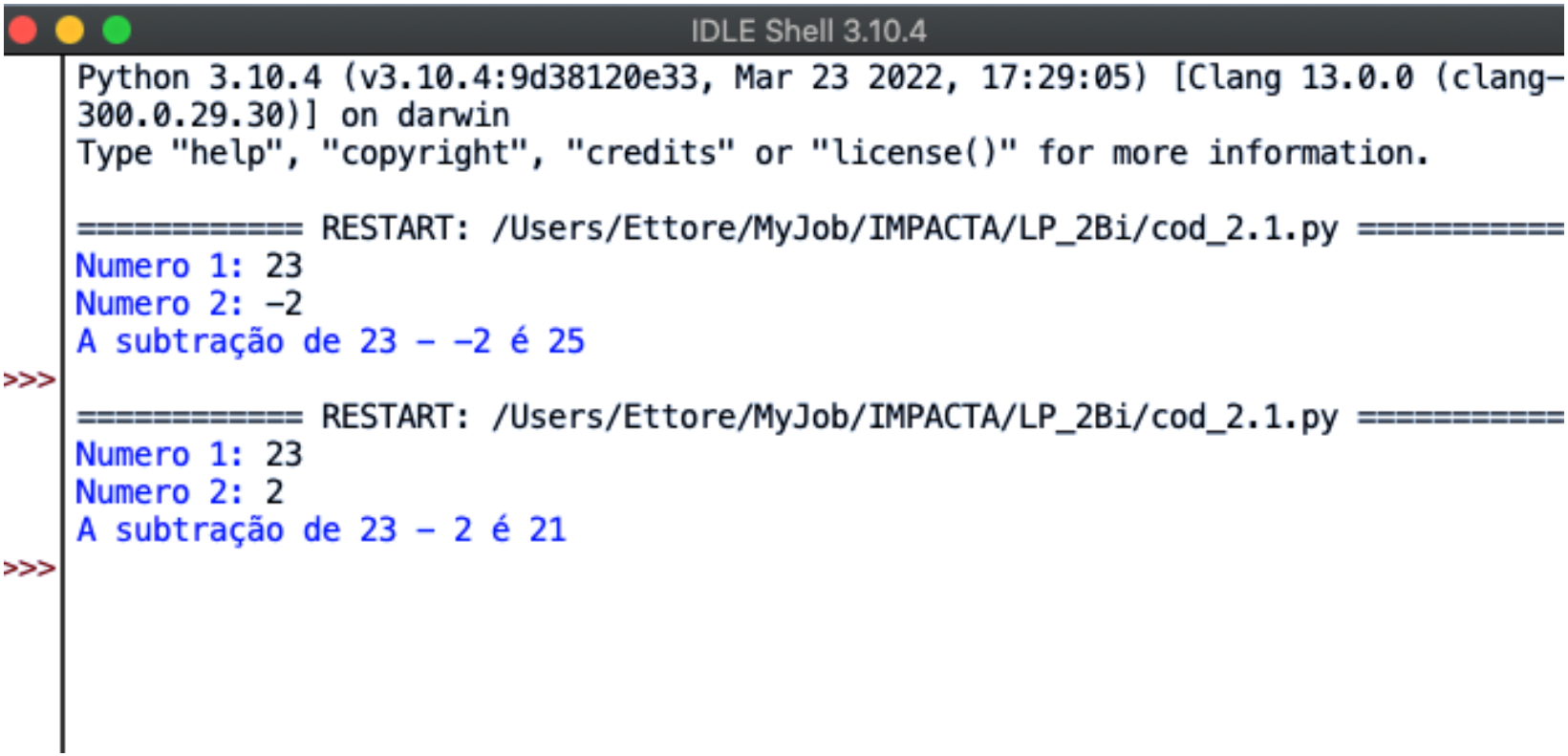
Por enquanto note que:

- Python **faz distinção** entre letras maiúsculas e minúsculas
- Antes de executar um programa você deve **salvá-lo**: Vá em File > Save, escolha o nome do arquivo e salve-o em uma pasta de sua preferência.
- Caso você esqueça de salvá-lo, ele não executará o código e pedirá para você fazer isso.

Executando o código fonte

Run > Run Module ou [F5]

Note que a execução é solicitada no editor mas é feita de fato no Shell



Conceitos básicos do Python

- O Python apresenta alguns tipos de **constantes (literais)** que devem ser usadas de acordo com a demanda de um dado problema. Usaremos quatro tipos inicialmente.
- **Números inteiros (int)**: valores numéricos sem ponto decimal

  Exemplos: 1, 123, -45
- **Números reais (float):** valores numéricos com casas decimais separadas pelo ponto decimal.

  Exemplos: 3.14, -2.34, 6.02e-23($= 6.02 \times 10^{-23}$). Note que e = 10 elevado à uma potência positiva ou negativa.
- **Valores booleanos (bool):** valores lógicos True e False
- **Textos (strings):** Cadeias de caracteres limitados por aspas ou apóstrofos. Exemplos: 'Oi', "Fulano é craque de bola! "

Operadores

Operadores são símbolos pré-definidos que realizam operações sobre sobre um ou mais operandos resultando em um valor cujo tipo dependerá do operador e dos operandos envolvidos.

Tipos de Operadores.

- Operador unário: realiza uma operação só com um operando
- Operador binário: realiza uma operação entre dois operandos
- Operador ternário: realizada uma operação com 3 operandos

Operadores aritiméticos

Tabela 2.1: Operadores aritméticos do Python.

Operador	Descrição	Exemplos
`+` (binário)	Soma o primeiro operando com o segundo.	`7 + 4 → 11` `1 + 2.0 → 3.0`
`-` (binário)	Subtrai o segundo operando do primeiro.	`15 - 5 → 10` `5 - 15 → -10`
`+` (unário)	Mantém o sinal do operando à direita. Observação: é a função identidade.	`+ 3 → 3` `+ (-3) → -3`
`-` (unário)	Inverte o sinal do operando à direita.	`- (3) → -3` `- (-7) → 7`
`*`	Multiplica o primeiro operando pelo segundo.	`3 * 8 → 24`
`/`	Quociente da divisão real do primeiro operando pelo segundo.	`9 / 2 → 4.5` `9.0 / 2 → 4.5`
`//`	Quociente da divisão inteira do primeiro operando pelo segundo.	`9 // 2 → 4` `9.0 // 2 → 4.0`
`%`	Resto da divisão inteira do primeiro operando pelo segundo.	`9 % 2 → 1` `9.0 % 2 → 1.0`
`**`	Exponenciação, eleva o primeiro operando ao segundo.	`3 ** 2 → 9`

Operadores aritméticos

Exercício 3: Resolva as operações com lápis, papel e calculadora e depois confira o resultado no Shell do Python.

a) 25.0 / 2

b) 25.0 // 2

c) 25.0 % 2

d) 1e4 + 500

e) 0 ** 0

e) 1e3 + 5

f) 0 / 2

g) 0 / 0

Variáveis

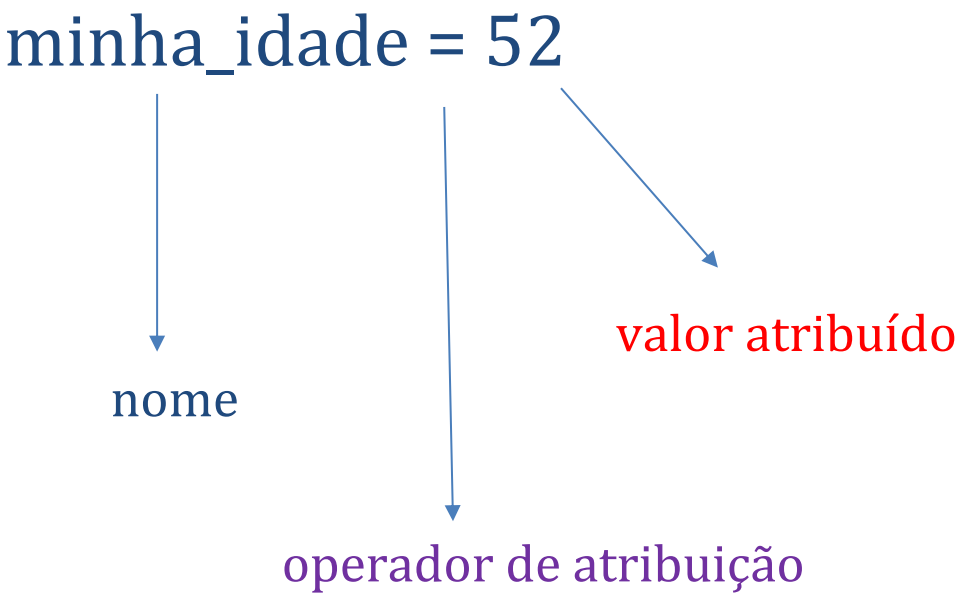
Uma **variável** é um espaço de memória associado a um **identificador** (nome) e serve para guardar valores que o programa pode acessar e modificar.

Toda variável possui um identificador **único** com o intuito de se evitar ambiguidades.

No Python uma variável é criada no **momento em que um valor é atribuído a um identificador válido.**

Variáveis

Exemplo



Uma variável só guarda um valor por vez, mas note que uma variável pode guardar **valores** e **tipos** diferentes em momentos diferentes de um programa.

- Cuidado: Python é case sensitive, ou seja: **meu_nome ≠ MEU_NOME**

Variáveis

- Para saber a classe de uma variável type(variável)

```
>>> a = True
>>> type(a)
<class 'bool'>
>>> a = 123
>>> type(a)
<class 'int'>
>>> a = 'boa noite'
>>> type(a)
<class 'str'>
>>> a = 5.46
>>> type(a)
<class 'float'>
>>>
```

- Lembrando: O tipo de dado está relacionado ao valor atribuído e não à variável que recebeu esse valor.

Identificadores

Um **identificador** é o **nome de uma variável** e tem regras para ser escrito.

- Deve conter **apenas** combinações de letras, dígitos e sublinhados
- **Nada de caracteres especiais como** @, $, %, ! etc
- **Não pode começar com um número**
- **Não pode ser uma palavra reservada** (keywords) do Python e para saber mais digite no shell help( ) e depois keywords. Para sair quit( ).
- Preferencialmente com letras minúsculas sem acentuação com palavras separadas por underline. Bom senso é o caminho, nomes curtos (abreviações podem não ser boas) que expressem o que vocês querem dizer no programa.

Comentários em Python

Comentários são ignorados na execução de um programa.

Para incluir um comentário use o caractere # no início da linha, ou após algum comando para explica-lo.

```
# Aumento do salário mínimo.
salario_20 = 1045.00  # salario em 2020
salario_21 = 1304.00  # salario em 2021
aumento_salario = salario_21 - salario_20
print('O salário mínimo aumento de:', aumento_salario, 'reais')
```

Python Tutor

É um ambiente gratuito que nos ajuda e entender o que ocorre durante a execução de um programa sob os pontos de vista de memória e saída de dados.

http://pythontutor.com

Exercícios

1. Escreva um programa que permita a entrada de duas notas (variáveis reais), Calcule sua média aritmética e exiba o resultado.

2. Baseado no programa anterior, modifique-o: O aluno tem 3 notas: P1, P2 e Trabalho (T). O programa deve pedir o valor dessas 3 notas e a média deve ser calculada da seguinte forma:

- A nota P1 tem peso 1.5
- A nota P2 tem peso 2.5
- O trabalho deve ter peso 1.0

Calcule e exiba a média ponderada deste aluno. $\bar{x}_p = \frac{\sum_{i=1}^{n} x_i p_i}{\sum_{i=1}^{n} p_i} = \frac{x_1 p_1 + x_2 p_2 + x_3 p_3}{p_1 + p_2 + p_3}$

Exercícios

3. Faça um programa que pergunte o ano em que a pessoa nasceu e que devolva a sua idade.

4. Faça um programa que pergunte a idade da pessoa em anos, meses e dias e que imprima quantos segundos essa pessoa já viveu. Considere que todos os meses tem 30 dias.

OBS. Notem que ainda não falamos em formatos de saída.

Este tópico será discutido em breve mas por enquanto usem

print(f ‘ <texto> {variavel:.nf}’) onde n é o numero de casas decimais para uma variável real.

FIM